EXP.NO: 6	Write a Servlet to demonstrate session tracking using HttpSession.
DATE:03/03/2026	

AIM:

To demonstrate session tracking using HttpSession in a Servlet-based web application. The exercise involves implementing a simple login system where a user can log in with their username and password, and their session will be tracked throughout their interaction with the web application.

ALGORITHM:

1. Input:

- The user will enter their **username** and **password** in a **login form**.
- The servlet will validate the credentials and create an **HTTP session** for the user if authentication is successful.

2. Steps for Implementing Session Tracking with HttpSession:

1. **Create the Login Form (HTML):**
   - Create an HTML form with two input fields for **username** and **password**.
   - Provide a **submit button** to send the data to the servlet for authentication.
2. **Create the Login Servlet:**
   - Write a Java servlet class to handle **login** requests.
   - The servlet will:
     - Retrieve the **username** and **password** from the form.
     - Authenticate the user by comparing the credentials to a stored value (e.g., hardcoded or from a database).
     - Create a new **HTTP session** for the user if the credentials are valid.
     - Set session attributes such as the user's **username** to track the session.
3. **Validate User Credentials:**
   - In the servlet, check if the **username** and **password** match the expected

values.

- If valid, create a session and store the **username** in the session.
- If invalid, redirect the user back to the login page with an error message.

4. **Create and Track User Session:**
   - Upon successful login, use the HttpSession object to store the session data.
   - Use the session.setAttribute("username", username) method to save the **username**.
   - Optionally, set session timeout using session.setMaxInactiveInterval(time_in_seconds) to manage session expiration.
5. **Redirect to a Protected Page:**
   - After a successful login, the user will be redirected to a **protected page** (e.g., a dashboard or home page).
   - The protected page should check if the user is authenticated by checking the **session attribute**.
6. **Logout and Session Invalidation:**
   - Provide a **logout feature** that invalidates the session using session.invalidate() when the user logs out.
   - After logout, redirect the user back to the login page.
7. **Session Expiration:**
   - If the session expires or the user manually logs out, the session will be invalidated, and the user will be redirected to the login page.

CODE:

```
<!DOCTYPE html>

<html>

<body>



<form action="LoginServlet" method="post">

  Username:
```

```
    <input type="text" name="user"><br><br>

    Password:
    <input type="password" name="pass"><br><br>

    <input type="submit" value="Login">
</form>

</body>
</html>
import java.io.*;
import jakarta.servlet.*;
import jakarta.servlet.http.*;
import jakarta.servlet.annotation.WebServlet;

@WebServlet("/LoginServlet")
public class LoginServlet extends HttpServlet {

    protected void doPost(HttpServletRequest request,
                          HttpServletResponse response)
            throws IOException {

        String user=request.getParameter("user");
        String pass=request.getParameter("pass");
```

```
        HttpSession session=request.getSession();


        if(user.equals("admin") && pass.equals("1234")) {
            session.setAttribute("username", user);
            response.getWriter().println("Login Successful");
        }
        else {
            response.getWriter().println("Invalid Login");
        }
    }
}
```

OUTPUT:

RESULT:

A simple login system is created where the user's session is tracked by a servlet to demonstrate session tracking using HttpSession.